Handoff 26: Python unit 2, part 2 (EDU-04: comparisons, Boolean logic, elif), and the unit complete

Date: 2026-10-27 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#PR), not merged, not deployed. **No database migration, no new dependency.** One additive API field (`complete` on each unit in `/api/catalog`). Unit 1 and lessons 1 to 3 are untouched.

> **How to use this file:** upload it with `build.md`, `docs/plan/teaching-design.md`, `docs/plan/lesson-feedback.md`, `docs/plan/browser-checks.md` and handoff 24. The pull request diff is the source of truth (`build.md` section 14). The appendix has every card, hint and check case word for word.

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 26 (EDU-04 part 2): the last three
lessons of Python unit 2 "Numbers and decisions" (py.compare,
py.boolean-logic, py.elif), which finish the unit, and a small change so
the lesson page and the home recommendation say correctly whether a unit
is complete. I'm attaching build.md, teaching-design.md,
lesson-feedback.md, browser-checks.md, handoff 24 and this handoff. Read
the appendix: check that the wording is calm, original and clear for a
grade 9 reader; that each check forces the idea it teaches (and, or,
elif) through its test cases and accepts other correct programs; that no
hint, tagged hint or similar example gives the answer; and that the
cards are accurate Python. Explain at a first-year computer engineering
level.
```

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done. No open PRs
PR #29 (EDU-04 part 1) merged into `main`?	**Yes**, 2026-10-27 20:59 UTC (`689fae9`)
Part 2 work on any branch?	**None**: no `py.compare`, `py.boolean-logic` or `py.elif` file on any local or remote branch
Where the work happened	A separate worktree, `../python-academy-u02b`, on a new branch `content/py-u02-part2` from `origin/main` at `61d57af` (after PR #30). The main folder was not switched. Not stacked on any PR
Part 1 on `main`	Identical to the part 1 branch I wrote (no code, content or test difference), so part 2 matches it exactly

2. Handoff record (build.md template)

Field	Value
Task	EDU-04 part 2: `py.compare`, `py.boolean-logic`, `py.elif`; finish unit 2 so the page and the recommendation say so correctly
Human goal	A learner can finish unit 2 live, and the site never claims a unit is complete, or points past it, when that is not true
Repository and branch	`torontojohn/python-academy`, branch `content/py-u02-part2`
Commits	`2b5cc85`: lessons, the unit-complete change, tests. Then docs and this handoff
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	A unit project or unit check; construct rule IDs; any change to unit 1 or lessons 1 to 3; `backlog.md` and `decision-log.md`
Data and compatibility	No migration. New lesson, task and badge IDs only. The API **adds** `complete` to each unit in `/api/catalog`; `/api/lessons` and the unit lesson routes are unchanged in shape. A page from before this PR ignores the new field; this page treats a missing field as "not known" and shows the "released so far" wording, which is never false
Deployment status	Not requested

Files changed

File	Why
`backend/content/py/u02/py.compare.json`, `py.boolean-logic.json`, `py.elif.json` (new)	The three lessons (appendix)
`backend/content/catalog.json`	Unit 2 lists all six lessons; its summary no longer says "come next"
`backend/app/content.py`	`Unit.more_lessons_coming` (optional, default `false`, allowed only on a released unit); `/api/catalog` adds `complete` per unit
`backend/app/recommend.py`	A unit with more lessons coming is not finished; the "coming soon" note names the unit and says what is true; `RULES_VERSION` 1 → 2
`frontend/app/catalog-data.ts`, `frontend/app/adventure.tsx`	The `complete` field; the lesson page says "Unit complete" only when `complete` is `true`
`backend/tests/test_unit2_part2.py` (new, 88 tests), `frontend/tests/unit2-part2.spec.ts` (new, 22 tests), `frontend/tests/fixtures/unit2-part2-api.json` (new)	Section 6
`backend/tests/test_unit2_lessons.py`, `frontend/tests/unit2.spec.ts`	Part 1 tests for a six-lesson unit; the hint audit now understands `elif` and `else` lines (section 5); the "released so far" ending tested with a unit still being written, and with an older API
`backend/tests/test_recommend.py` (2 new tests), `test_academy.py`, `test_content.py`, `frontend/tests/academy.spec.ts`; fixtures `catalog_v1.json`, `lesson-units-api.json`, `unit2-api.json`	Six lessons in unit 2 (11 in the course), the `complete` field, rules version 2, the new note
`backend/content/README.md`, `docs/plan/course-catalog.md` (4.3), `docs/plan/browser-checks.md`	The new field, the note wording, unit 2 done

File	Why
`docs/handoffs/2026-10-27-26-unit2-part2.md` and `.pdf`	This handoff

3. How it works

The story. The riddle keeper has a secret number and the player guesses. Lesson 4 says whether the guess is too high, too low or right; lesson 5 whether it is close (no more than 2 away) or out of range (outside 1 to 100); lesson 6 gives points: 10 for exactly right, 5 for close, 1 for within 10, 0 otherwise.

Each lesson has the five cards of part 1. Answers are `True`/`False` lines or a number, so wording is never judged and any correct program passes. Support fades: lesson 4's starter asks for both numbers; lesson 5's has two comments; lesson 6's, two shorter ones. Predict-then-run on cards 3 and 4 in every lesson; fix-the-bug as far as five cards allow: - **Lesson 4:** the predict card is the `>=` boundary question. - **Lesson 5:** the "Try it" card turns `and` into `or` and asks when the answer changes. - **Lesson 6:** the "Try it" card turns `elif` into `if` and asks why two lines print.

The constructs are forced by behaviour, not by a construct rule (no `require`, no new naming):

Lesson	The trap	The case that catches it
`py.boolean-logic`	Joining the "close" limits with `or` is always `True`	A guess 3 away (`just-outside`)
`py.boolean-logic`	Joining "out of range" with `and` is always `False`	A guess of 0 (`below-range`)
`py.elif`	Separate `if` statements: an exactly right guess is also close and within 10, so it prints three scores (or keeps the last one, 1)	The exact case, visible first (`exact`)
`py.elif`	Conditions in the wrong order	The same exact case

A correct program written with separate `if`s that print one score at the end still passes (it is in "Other ways"): the check judges behaviour, not style.

Unit complete: the smallest additive way. A content field, `more_lessons_coming`, on a unit in `catalog.json`. It defaults to `false`, so nothing in the real catalog sets it now that unit 2 is whole. `/api/catalog` adds `complete` = released and not `more_lessons_coming`.

Where	Complete unit	Unit still being written
Lesson page, after the last lesson	**"Unit complete. You finished every lesson in Numbers and decisions. See the course for what comes next."**	"You finished every lesson released so far…" (as in part 1); also shown if the API is older and has no field
Home recommendation note	"Python Foundations unit 3, Repeat and explore, is next on your path. Its lessons are not written yet."	"Python Foundations unit 2, Numbers and decisions, has more lessons on the way. You have finished every lesson in it so far." The unit after it is never named

Unit 1's page keeps its own ending ("You made an adventure."), unchanged.

4. Decisions and why

Decision	Options considered	Reason	Who decided
No construct rules; the cases force `and`, `or` and `elif`	Add `uses-elif`, `uses-and` rules	Each lesson could be checked fairly by behaviour, so no naming was invented (the owner's rule). A forced construct would also reject correct alternatives (chained comparisons, `abs()`, nested `if/else`)	Implementer, within the owner's rule
`more_lessons_coming` on the unit (content) and `complete` in `/api/catalog` (API)	A count of planned lessons; the page comparing with the plan; leaving the "released so far" wording forever	One boolean is the smallest fact that answers the question; it is set by the person releasing a part; the API field is additive. A planned-lesson count would need keeping in step with the plan	Implementer (**assumption 1**: the name)
A missing `complete` reads as not known, with the "released so far" wording	Read missing as complete	During a deploy an older API has no field; the "released so far" wording is never false	Implementer
The note for an unwritten unit says "Its lessons are not written yet." instead of "is coming soon"	Keep "coming soon"; drop the note	You asked for no more "unit 3 is next and coming soon" while unit 3 does not exist, and to say what is true. Unit 3 is planned and named in the catalog, but has no lessons. The course-level note ("Hackathon Ready is next on your path and is coming soon.") is unchanged	Owner rule; implementer wording (**assumption 2**)
`RULES_VERSION` 1 → 2	Keep 1	"Unit done" changed meaning for a unit still being written; the version tells anyone reading saved or logged recommendations which rules made them	Implementer
Outputs are `True/False` or a number	Words such as "too high"	Exact-output checks judge wording too; `True/False` is what a comparison gives, which is what the lessons teach	Implementer
Lesson 6's points use `and` (lesson 5) for "within 2" and "within 10"	`abs()` only	It joins the unit's ideas; `abs()` is shown as another way after passing	Implementer
One practice badge per lesson	No per-lesson badge (`course-catalog.md` 6.2)	Your rule, as in part 1 (handoff 24 question 6 is still open)	Owner

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
The hint audit could not read `elif …:` or `else:` in a hint (they do not parse on their own), so lesson 6's approach was not tested	`test_the_audit_finds_the_code_in_the_hints[py.elif]` failed	The audit now parses those after an `if` block and assembles them into a valid chain (`assemble()` in `test_unit2_lessons.py`); part 1's results are unchanged

Bug or near-miss	How it was found	Fix
A tagged hint that could never be shown (`ValueError` in lesson 5: no case has an empty or non-number reply)	`test_every_tagged_hint_is_for_a_rule_or_error_this_lesson_can_report`	Removed before the first commit
Near-miss: part 2's hidden outputs repeat visible ones (`True`, `False`, `False`; `10`), so a search of answers for them proves nothing	Writing the leak test	The test checks the plan's structure (no `expected` on any hidden case) and that hidden failures never carry expected lines; break B3 shows it fails
Near-miss: 9 "CSP" console lines in the no-mock run	As in handoff 24	My own axe injection, one per scan; the app logs none, and the browser tests assert no violations

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**541 passed, 11 skipped** (451 before, plus 88 in `test_unit2_part2.py` and 2 in `test_recommend.py`)
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**552 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**173 passed** (150 before, plus 22 in `unit2-part2.spec.ts` and 1 more ending variant in `unit2.spec.ts`)
No-mock run: real API (uvicorn), production build, real Pyodide, 8 programs per lesson, the six-lesson ending, and axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA, best practices) on every card, result, the course page, the catalog and the ending, at 1280 px and 375 px	Every result as expected; the ending says "Unit complete."; **0 axe violations**; no sideways scrolling

What the tests cover

Required test	Where
Content validation	Each lesson: browser check v1, 3-second limit, at least 4 visible and 2 hidden cases, no `require`/`forbid`, three hints, a similar example, two other ways, a new practice badge; fading; `more_lessons_coming` refused on a coming-soon unit
What the cards say is true	Every worked example's output; 12 claims on the cards and hints, and the three predict answers, run in Python; the error hints' TypeError and SyntaxError claims
Correct programs pass	The reference, 3 creative solutions (chained comparisons, `abs()`, `range`, nested `if`/`else`, reversed `elif` order) and the 2 other ways per lesson
Wrong programs fail with the right rule	34 samples, every case rule (hidden too) with one, including separate `if`s, `or` for `and`, `and` for `or`, off-by-one edges, hard-coded and visible-only answers
Hint audit	Code in the hints, tagged hints and similar example, alone and in every combination, with and without the starter and after the two input lines; and again with the hints' names swapped for the challenge's
Every rule has a message and a sample	`test_every_case_rule_has_a_wrong_sample_and_a_message`
Hidden output never sent	Plans and answers (backend and browser), and ENG-03's own leak tests
End to end with real Pyodide, each lesson	Every card (a wrong then right prediction, a "Try it" run); a pass with the badge and other ways; needs_work with the case's tagged hint first (lesson 6: the separate-ifs answer printing 10, 5, 1); a Python error with the friendly explanation, line and Python's message (NameError, SyntaxError, IndentationError); an endless loop stopped at 3 seconds; an unsupported import; a hidden case named by description only
XP and badges once	300 XP and 3 badges for part 2, nothing for a failed attempt, nothing twice; part 1's test unchanged
Unit 1 unchanged	`unit1-unchanged.spec.ts` passes unchanged; `/api/lessons` equals the frozen fixture
The unit-complete ending	All six lessons passed in one session: "Unit complete.", 600 XP, 6 of 6 badges; a unit still being written and an older API: "released so far"
Recommendation	After unit 2: the unit 3 note; part of unit 2 done: continue with lesson 4; a unit marked `more_lessons_coming`: "has more lessons on the way", and unit 3 never named
axe at desktop and phone	The no-mock run above, plus the phone and reduced-motion browser test

Mocks: the browser tests mock the API with recorded **real** answers (`test_the_recorded_answers_are_the_real_apis`, for both parts). Python, the worker, the runner and the page are real. The no-mock run used nothing fake.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Lesson 4's approach hint spells out the answer	**5**: the hint audit, the renamed-hint audit, "the audit finds the code", and both recordings tests

What I broke	Tests that failed
B2. Lesson 6's check accepts a separate-`if` chain (only the non-overlapping cases kept)	**16**: eight wrong samples now pass (separate `if`s, wrong order, edges, visible-only), the separate-ifs test, the content check (too few cases), hard-coding, the case-rule samples, the XP test, the renamed audit and both recordings tests
B3. The check plan sends hidden cases' expected output	**12**: ENG-03's two leak tests, all six unit 2 leak tests, the plan-structure test, and the three recordings tests
B4. The lesson page always says "Unit complete"	**2**: both "unit still being written" browser tests (the content field and the older API)
B5. The recommendation treats a unit still being written as finished	**1**: `test_a_unit_still_being_written_is_not_finished_and_says_more_lessons_are_coming`

7. Known limitations

- **No real unit uses `more_lessons_coming` now**; it is tested with a modified catalog. The next unit released in parts should set it.
- **"Unit complete" is a page message, not a record.** No unit badge or unit check exists yet (ASSESS-02 onwards).
- **Lesson 6 scores only by distance**; "too high" and "too low" words are lesson 4's `True`/`False` lines, by design (section 4).
- **The home note** cannot be seen in the browser tests with a real account (they mock the recommendation); the backend tests check its words.
- **Not tested with a real screen reader**, or on Safari or Firefox. Reading level was written for grade 9, not measured with a formula.

8. Live checks for you (after merging and deploying)

You have one test account; your sister can use `/preview` with no account.

- [ ] **The catalog.** `/courses/py`: unit 2 lists six lessons, and its summary ends "…comparisons, Boolean logic and elif."
- [ ] **Lesson 4.** In "Too high or too low?", card 5: add `print(guess >= secret)`, `print(guess < secret)`, `print(guess == secret)` and run. "Not yet. For a secret of 7 and a guess of 7, line 1 should be "False"…". The hint button offers a hint about this check. Change `>=` to `>`: "Check passed".
- [ ] **Lesson 5.** In "Close enough?", join the close limits with `or` and run: "Not yet" for a guess of 53, and a hint about `or`.
- [ ] **Lesson 6.** In "The riddle keeper's verdict", write three separate `if` statements that each print a score, then run: the Output panel shows 10, 5 and 1 for the exact guess, and the hint explains `elif`. Rewrite it with `elif`: "Check passed".
- [ ] **The ending.** As your test student, finish all six lessons of unit 2: "Unit complete. You finished every lesson in Numbers and decisions."
- [ ] **The home page** after that: "Python Foundations unit 3, Repeat and explore, is next on your path. Its lessons are not written yet."

- [ ] **XP.** 100 more XP and one badge per lesson; passing again adds nothing.
- [ ] **Unit 1 unchanged.** "Open the story" looks and behaves as before.
- [ ] **For your sister:** are lessons 5 and 6 clear? "Close" and "within 10" are on either side of the secret; is that obvious from the prompts?
- [ ] **No CSP reports** in DevTools → Console.

9. Questions for you

1. **Assumption 1, the field name** `more_lessons_coming` (content) and `complete` (API). OK?
2. **Assumption 2, the note** "Its lessons are not written yet." for a planned unit with no lessons, while a planned course keeps "is coming soon". Same wording for both? *Recommended, not yet confirmed: keep as is; a course and a unit are different promises.*
3. **Unit 2 is complete:** what next, a unit check for unit 2 (ASSESS-02) or unit 3? (No recommendation; it depends on your priorities.)
4. **Still open from handoff 24:** question 6 (one badge per lesson against `course-catalog.md` 6.2).

10. For your docs PR (not edited here)

Where	What
Backlog EDU-04	in_review for part 2 (this PR, handoff 26); part 1 merged (#29)
Stage table	"26. EDU-04 part 2 | Unit 2 lessons 4 to 6, unit complete | in_review (handoff 26)"
Decision log	A chapter from sections 4 to 6 of this handoff

11. Next concrete action

Review and merge, run the live checks, then choose between a unit 2 check and unit 3 (question 3).

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-u02b status --short   # expect nothing, or only ?? frontend/AGENTS.md
rm -f ../python-academy-u02b/frontend/AGENTS.md   # generated by Next.js, not part of the project
git worktree remove ../python-academy-u02b     # deletes only that folder; refuses if anything is
uncommitted
git worktree prune
git branch -d content/py-u02-part2             # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that). The part 1 worktree (`../python-academy-u02`) and the ID-01 worktree (`../python-academy-id01`) can be removed the same way, since PRs #29 and #30 are merged.

Appendix. Every card, hint and check case, word for word

Generated from the three content files. Hidden cases' expected output is listed here for review only; it never reaches the browser.

A.1 `py.compare`: Too high or too low?

Concept: Comparisons · < > <= >= == !=. Description: Compare the player's guess with the riddle keeper's secret number. Badge: **Riddle judge** (`py.badge.riddle-judge`, practice).

Card	Text
1. Discover: Compare two numbers	A comparison asks Python a yes-or-no question about two values. 7 > 5 is True, and 7 < 5 is False. True and False are values too, so print(7 > 5) shows True. There are six comparisons: < (less than), > (greater than), <= (less than or equal to), >= (greater than or equal to), == (equal to) and != (not equal to). Two equals signs compare; one equals sign stores a value. 5 <= 5 is True, but 5 < 5 is False.
2. Worked example	`# The lantern needs at least 3 drops of oil` / `oil = 5` / `print(oil >= 3)` / `# Is the tank completely full at 10?` / `print(oil == 10)` / `# Is it empty?` / `print(oil < 1)` Replies: none. Output: `True` / `False` / `False` oil is 5, so oil >= 3 is True. It is not 10, and it is not less than 1, so the other two lines show False.
3. Predict	If oil were 3, what would print(oil >= 3) show? Options: **True** (answer); False Why: >= means greater than or equal to. 3 is equal to 3, so the answer is True. print(oil > 3) would show False.
4. Try it	Set oil to 3, then to 0. Before each run, predict all three lines, then run and compare. Then change >= to > and see which line changes.
5. Challenge	The riddle keeper has a secret number, and the player makes a guess. The starter asks for both. Print three lines, each True or False: is the guess too high, is it too low, and is it exactly right? Print just the three answers. The check tries several guesses, including some you can't see. Starter: `# These two lines ask for the secret number and the guess` / `secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `# Print True or False: too high, then too low, then exactly right`

Hint	Text
A nudge	Each line is one comparison between guess and secret. print() of a comparison shows True or False.
A bigger hint	Look at the comment. Too high means the guess is bigger than the secret, too low means it is smaller, and right means they are equal. Comparing for equal uses two equals signs.
The approach	A pattern with other names: print(height > limit) shows True when height is over the limit. Your three lines compare guess with secret in three different ways.
About `NameError`	Use the names from the starter, secret and guess, spelled exactly the same.
About `TypeError`	Inside print(), compare with == (two equals signs). A single = inside print() makes Python stop with a TypeError.
About `SyntaxError`	Each comparison sign is written together with no space inside it: >=, <=, == and !=. Check the line Python points to.
About `py.compare.case.higher`	With a secret of 7 and a guess of 9, the guess is too high, so the lines are True, False, False. Check the order of your three lines.

Hint	Text
About `py.compare.case.equal`	When the guess equals the secret, it is neither too high nor too low: False, False, True. If your first line says True, you may have used >= where > belongs.
About `py.compare.case.below-zero`	Your comparisons need to work for every number, including ones below zero. Compare guess with secret instead of checking for particular numbers.
About `py.compare.case.large`	Compare guess with secret, not with numbers typed into the code, so your program works for every secret.
Similar example	`speed = 45` / `limit = 50` / `print(speed > limit)` / `print(speed != limit)` A speed of 45 is not over the limit of 50, so the first line is False, and the two are not equal, so the second line is True. Your challenge compares a guess with a secret number in three ways.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `print(secret < guess)` / `print(secret > guess)` / `print(secret == guess)` A comparison can face the other way: secret < guess means the same as guess > secret.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `too_high = guess > secret` / `too_low = guess < secret` / `print(too_high)` / `print(too_low)` / `print(not too_high and not too_low)` True or False can be stored in a variable. The last line uses not and and, which the next lesson explains.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`higher`	visible	a secret of 7 and a guess of 9	'7', '9'	`True` / `False` / `False`
`lower`	visible	a secret of 7 and a guess of 4	'7', '4'	`False` / `True` / `False`
`equal`	visible	a secret of 7 and a guess of 7	'7', '7'	`False` / `False` / `True`
`one-above`	visible	a secret of 20 and a guess of 21	'20', '21'	`True` / `False` / `False`
`below-zero`	hidden	a secret below zero	'-5', '-8'	`False` / `True` / `False`
`large`	hidden	a large secret	'1000', '999'	`False` / `True` / `False`

A.2 `py.boolean-logic`: Close enough?

Concept: Logic · and, or, not. Description: Decide whether a guess is close or out of range by joining comparisons together. Badge: **Close call** (`py.badge.close-call`, practice).

Card	Text
1. Discover: Join two questions into one	and, or and not combine True and False. A and B is True only when both are True. A or B is True when at least one of them is True. not flips True to False, and False to True. Each side of and or or is a whole comparison: write x > 0 and x < 10, not x > 0 and < 10. Python works out the comparisons first, then not, then and, then or. Use parentheses when you want to be sure of the order.
2. Worked example	`# The bridge is safe for 2 to 6 travellers` / `travellers = 7` / `print(travellers >= 2 and travellers <= 6)` / `# Too few or too many?` / `print(travellers < 2 or travellers > 6)` / `# Flip the first answer with not` / `print(not (travellers >= 2 and travellers <= 6))` Replies: none. Output: `False` / `True` / `True` 7 is at least 2 but more than 6, so the and line is False. The or line needs only one side to be True, and 7 > 6 is. not flips the first answer from False to True.

Card	Text
3. Predict	If travellers were 2, what would the first print() show? Options: **True** (answer); False Why: 2 >= 2 is True and 2 <= 6 is True. With and, both sides are True, so the whole line is True.
4. Try it	Try travellers = 2, then 6, then 0, and predict the three lines before each run. Then change and to or in the first print(). Find a number of travellers where the answer changes, and explain why.
5. Challenge	The riddle keeper's secret number is always from 1 to 100. Ask for the secret and the guess, as whole numbers. Print three lines, each True or False: is the guess close (no more than 2 away from the secret, on either side); is the guess out of range (below 1 or above 100); and is the guess not close? Print just the three answers. Starter: `# Ask for the secret number, then the guess, as whole numbers` / `# Print True or False: close, out of range, not close`

Hint	Text
A nudge	Three lines, three joined questions. Close needs two comparisons joined with and. Out of range needs two joined with or. The last line flips the first one with not.
A bigger hint	Close means the guess is at least secret - 2 and at most secret + 2. Out of range means below 1 or above 100. Check whether each limit should use < or <=.
The approach	A pattern with other names: print(depth >= low and depth <= high) checks that depth sits between two limits. Putting not ( ) around a question flips its answer.
About `SyntaxError`	Each side of and and or needs a whole comparison, like guess >= 1 and guess <= 100. Writing guess >= 1 and <= 100 makes Python stop.
About `NameError`	Give secret and guess their values from input() before you compare them, and spell them the same each time.
About `py.boolean-logic.case.just-outside`	A guess of 53 is 3 away from 50, so it is not close. If your first line says True, check whether you joined the two limits with or: or is True when either side is, and here one side always is.
About `py.boolean-logic.case.edge-above`	52 is exactly 2 away from 50, which still counts as close. Check that your limits use <= and >=, so the edge is included.
About `py.boolean-logic.case.below-range`	0 is below 1, so it is out of range: True. Out of range needs or, because a guess can't be below 1 and above 100 at the same time.
About `py.boolean-logic.case.top-edge`	Check both ends of the range: a guess equal to the highest or lowest allowed answer is still in range.
About `py.boolean-logic.case.bottom-edge`	A guess equal to the lowest allowed answer is still in range, and a guess 1 away from the secret is close. Check each < against <=.
Similar example	`temperature = 18` / `print(temperature >= 15 and temperature <= 25)` / `print(not temperature > 30)` 18 is between 15 and 25, so the first line is True, and 18 is not over 30, so the second line is True too. Your challenge checks a guess against limits that come from the secret number.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `close = secret - 2 <= guess <= secret + 2` / `print(close)` / `print(guess < 1 or guess > 100)` / `print(not close)` Python lets you chain two comparisons, which reads like maths: low <= x <= high.

Hint	Text
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `distance = guess - secret` / `close = distance >= -2 and distance <= 2` / `print(close)` / `print(not (1 <= guess <= 100))` / `print(not close)` Work out how far away the guess is first. Out of range is the same as not in range.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`edge-above`	visible	a secret of 50 and a guess of 52	'50', '52'	`True` / `False` / `False`
`just-outside`	visible	a secret of 50 and a guess of 53	'50', '53'	`False` / `False` / `True`
`edge-below`	visible	a secret of 50 and a guess of 48	'50', '48'	`True` / `False` / `False`
`below-range`	visible	a secret of 50 and a guess of 0	'50', '0'	`False` / `True` / `True`
`above-range`	visible	a secret of 50 and a guess of 101	'50', '101'	`False` / `True` / `True`
`top-edge`	hidden	a guess at the top of the range	'99', '100'	`True` / `False` / `False`
`bottom-edge`	hidden	a guess at the bottom of the range	'2', '1'	`True` / `False` / `False`

A.3 `py.elif`: The riddle keeper's verdict

Concept: Decisions · elif. Description: Give points for a guess by how close it is, choosing exactly one branch. Badge: **Fair verdict** (`py.badge.fair-verdict`, practice).

Card	Text
1. Discover: One choice out of many	if and else choose between two paths. elif, short for "else if", adds more. Python checks each condition from the top and runs only the first block whose condition is True. It skips the rest. Order matters when conditions overlap. A guess that is exactly right is also close, so check for exactly right first. else catches everything the conditions above it did not. Each elif line ends with a colon, and its block is indented by four spaces.
2. Worked example	`# Choose a cloak for the weather` / `temperature = 12` / `if temperature >= 25:` / `print("light cloak")` / `elif temperature >= 10:` / `print("wool cloak")` / `else:` / `print("fur cloak")` Replies: none. Output: `wool cloak` 12 >= 25 is False, so Python moves on. 12 >= 10 is True, so it prints wool cloak and skips else. With 30, only light cloak prints, even though 30 >= 10 is True as well.
3. Predict	If temperature were 30, what would the program print? Options: **light cloak** (answer); light cloak, then wool cloak; wool cloak Why: Python runs only the first block whose condition is True. 30 >= 25 is True, so light cloak prints, and the elif and else are skipped.
4. Try it	Predict the cloak for 25, 10 and 9, then run each one. Next, change elif to if, run with 30 and compare: why do two lines print now? Reset code puts the example back.
5. Challenge	The riddle keeper gives points for a guess: 10 if it is exactly right, 5 if it is close (no more than 2 away), 1 if it is no more than 10 away, and 0 otherwise. Ask for the secret number and the guess, then print the points. Print just the number. Starter: `# Ask for the secret number and the guess` / `# Print 10, 5, 1 or 0: check the closest case first`

Hint	Text
A nudge	One guess gets one score. Work out how close the guess is to the secret, then print exactly one of 10, 5, 1 or 0.
A bigger hint	Look at the order of your checks. A guess that is exactly right is also close, so ask about exactly right first, then close, then within 10, and let else give 0.
The approach	A pattern with other names: if age >= 18: with an indented line, then elif age >= 13: with another, then else: with a last one. Your conditions compare the guess with the secret.
About `SyntaxError`	if, elif and else lines end with a colon, and comparing for equal uses ==. An elif must come straight after an if block or another elif block.
About `IndentationError`	The line under each if, elif and else starts with four spaces. Press Tab at the start of the line to add them.
About `NameError`	Ask for both numbers with input() and give them names before your if line, spelled the same each time.
About `py.elif.case.exact`	An exactly right guess scores 10, printed once. Separate if statements can all run; in an if and elif chain only the first match does. Put the exact check first.
About `py.elif.case.close`	42 is 2 away from 40, so it scores 5. If you printed 1, check the order of your conditions. If you printed two numbers, use elif so only one branch runs.
About `py.elif.case.near-edge`	50 is exactly 10 away from 40, which still scores 1. Check that this limit uses <= or >=, so the edge counts.
About `py.elif.case.far`	51 is 11 away, more than 10, so it scores 0. else is the place for everything the other branches did not catch.
About `py.elif.case.close-below`	Guesses below the secret count too: close and within 10 mean on either side. Use and to check both limits, or work out how far away the guess is.
About `py.elif.case.far-below`	A guess far below the secret scores 0. Check that your within-10 test has a lower limit as well as an upper one.
About `py.elif.case.exact-again`	Compare guess with secret, not with numbers typed into the code, so your program works for every secret.
Similar example	`age = 9` / `if age < 5:` / `print(0)` / `elif age < 13:` / `print(4)` / `else:` / `print(8)` A ticket price by age: under 5 is free, under 13 costs 4, and everyone else pays 8. A 9 year old matches only the second condition, so only 4 prints. Your challenge has four outcomes and compares two numbers.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `distance = abs(guess - secret)` / `if distance == 0:` / `print(10)` / `elif distance <= 2:` / `print(5)` / `elif distance <= 10:` / `print(1)` / `else:` / `print(0)` abs() gives how far apart two numbers are, ignoring which one is bigger.
Other way, after passing	`secret = int(input("Secret number? "))` / `guess = int(input("Your guess? "))` / `points = 0` / `if guess >= secret - 10 and guess <= secret + 10:` / `points = 1` / `if guess >= secret - 2 and guess <= secret + 2:` / `points = 5` / `if guess == secret:` / `points = 10` / `print(points)` Separate if statements work here because each later check replaces the points with a bigger score, and only one number is printed at the end.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`exact`	visible	a secret of 40 and a guess of 40	'40', '40'	`10`
`close`	visible	a secret of 40 and a guess of 42	'40', '42'	`5`
`near`	visible	a secret of 40 and a guess of 37	'40', '37'	`1`
`near-edge`	visible	a secret of 40 and a guess of 50	'40', '50'	`1`
`far`	visible	a secret of 40 and a guess of 51	'40', '51'	`0`
`exact-again`	hidden	another secret, guessed exactly	'7', '7'	`10`
`close-below`	hidden	a close guess below the secret	'100', '98'	`5`
`far-below`	hidden	a guess far below the secret	'12', '-3'	`0`